

DROSOPHILA-LANG: UN LENGUAJE DE PROGRAMACIÓN ESOTÉRICO QUE SE EJECUTA EN EL CEREBRO DE UNA MOSCA

VIOREL STAVARA

RESUMEN. DROSOPHILA-LANG es un lenguaje de programación esotérico cuyos programas se ejecutan sobre el mapa completo del cerebro de la mosca de la fruta que publicó el proyecto FlyWire: 138 639 neuronas y 2 700 513 conexiones reales. Cada paso del programa se asigna a una neurona real, cada variable es la fuerza de una conexión que la dopamina refuerza o debilita, y el programa termina cuando dispara la neurona con la que la mosca huye. Este documento define el lenguaje y el modelo de neurona con el que se ejecuta, demuestra que es Turing-completo por reducción a las máquinas de Minsky, acota la probabilidad de que el ruido neuronal cambie el resultado y discute qué parte del cálculo hace de verdad el cerebro de la mosca.

ÍNDICE

1. Introducción	2
2. La mosca en cinco minutos	2
3. El lenguaje	3
3.1. Variables y estados	4
3.2. Las órdenes	4
3.3. Variables y datos	5
3.4. Un ejemplo completo: la suma	5
4. Cómo se ejecuta	5
4.1. Del comando al resultado	5
4.2. Neuronas reales, cables añadidos	6
4.3. El modelo de neurona	6
4.4. La memoria	7
4.5. Cómo elige las neuronas el intérprete	8
4.6. Dos capas de actividad	8
4.7. Cómo termina	8
5. ¿Por qué es Turing-completo?	8
5.1. Máquinas de Minsky	9
5.2. La traducción	9
5.3. El cerebro es finito	10
6. Resultados	11
6.1. Programas de ejemplo	11
6.2. Dónde vive el programa	11
6.3. El cerebro reacciona	12
6.4. Comprobación de la demostración	12
6.5. Robustez frente al ruido	13
7. Reflexión	13
7.1. ¿Calcula la mosca?	13

Fecha: 30 de septiembre de 2026. Código: github.com/stavara/drosophila-lang, commit c0a58bf.

7.2. Programar no es entrenar	14
7.3. ¿Y en una mosca viva?	14
7.4. ¿Dónde está el programa?	14
7.5. Qué hace falta para calcular	14
7.6. La anatomía como gramática	14
7.7. Limitaciones	14
8. Conclusiones	15
Apéndice A. Gramática	15
Apéndice B. Programas de ejemplo	16
Apéndice C. Cómo ejecutarlo	16
Referencias	18

1. INTRODUCCIÓN

Un lenguaje de programación es una forma de escribir órdenes que una máquina sigue al pie de la letra. Un lenguaje *esotérico* es uno inventado para explorar una idea, no para usarlo en el trabajo: suele tener muy pocas instrucciones o una forma extraña de ejecutarse, y precisamente por eso obliga a preguntarse qué hace falta, como mínimo, para que algo pueda calcular.

La idea de DROSOPHILA-LANG es que quien ejecute las órdenes sea el cerebro de una mosca. En 2024, el proyecto FlyWire publicó el primer mapa completo del cerebro de un animal adulto: todas las neuronas de la mosca de la fruta y todas las conexiones entre ellas [1], junto con el tipo de cada neurona [2]. Ese mismo año, Shiu *et al.* mostraron que, simulando ese mapa en un ordenador, se reproducen respuestas reales de la mosca [3]. DROSOPHILA-LANG usa esos datos: cada pieza de un programa se asigna a una neurona real con un papel parecido, y ejecutar el programa es simular el cerebro paso a paso.

La idea de que las neuronas pueden calcular es antigua: en 1943, McCulloch y Pitts ya describieron la neurona como una puerta lógica [4]. Lo nuevo aquí es hacerlo sobre un cerebro concreto, con sus tipos de neurona y sus limitaciones, y comprobar hasta dónde llega.

El documento sigue el recorrido de un programa. El Apartado 2 explica la parte del cerebro que usa el lenguaje; el Apartado 3, cómo se escribe un programa; el Apartado 4, el modelo de neurona con el que se ejecuta; el Apartado 5 demuestra que el lenguaje puede calcular cualquier cosa; el Apartado 6 muestra qué pasa al ejecutarlo de verdad; y el Apartado 7 discute qué parte del cálculo hace realmente la mosca.

2. LA MOSCA EN CINCO MINUTOS

Para entender el lenguaje basta con conocer un recorrido del cerebro de la mosca: el que va desde que la antena detecta un olor hasta que la mosca responde. La Figura 1 lo resume, junto con el papel que cada neurona desempeña en DROSOPHILA-LANG.

- (1) **Receptores olfativos (ORN).** Las neuronas de la antena detectan moléculas del aire. Todas las que detectan el mismo tipo de olor envían sus axones al mismo punto del lóbulo antenal, una pequeña esfera llamada *glomérulo*.¹

¹La mosca tiene unos cincuenta glomérulos con nombres como DA1, DM1 o VA2, que indican su posición (dorsal, anterior, ventral...). El glomérulo DA1, por ejemplo, responde a la feromona cVA [5].

- (2) **Neuronas de proyección (PN).** Desde cada glómulo, unas pocas neuronas de proyección llevan la señal hacia dos centros superiores: el cuerpo en champiñón y el cuerno lateral.
- (3) **Cuerpo en champiñón: células de Kenyon (KC) y MBON.** Es el centro de aprendizaje olfativo. Unas cinco mil células de Kenyon reciben la señal de las neuronas de proyección y la transmiten a unas pocas neuronas de salida, las MBON (del inglés *mushroom body output neurons*). La fuerza de cada sinapsis $KC \rightarrow MBON$ puede cambiar: así se guarda lo que la mosca aprende de un olor [6].
- (4) **Neuronas dopaminérgicas (PAM y PPL1).** Liberan dopamina sobre las sinapsis $KC \rightarrow MBON$ y modifican su fuerza. Hay dos familias que actúan en compartimentos distintos, llamadas PAM y PPL1 por la posición de sus cuerpos celulares (*protocerebral anterior medial* y *protocerebral posterior lateral*) [6].²
- (5) **Cuerno lateral (LH).** Recibe también la señal olfativa y contiene neuronas locales que la combinan. DROSOPHILA-LANG las usa como compuertas lógicas.
- (6) **Salidas motoras.** La motoneurona MN9 extiende la probóscide, la «trompa» con la que la mosca come [8]; y la Fibra Gigante (DNp01; las siglas DN vienen de *descending neuron*) desencadena la huida, el salto con el que la mosca escapa de un peligro [9]. DROSOPHILA-LANG usa además otras motoneuronas del cerebro y la neurona descendente DNa02 como salida de datos.

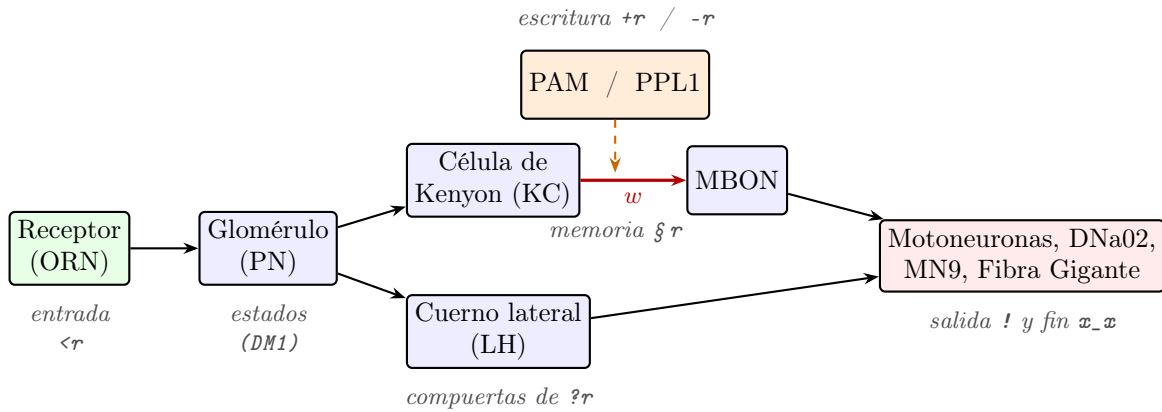


FIGURA 1. Recorrido del olor en el cerebro de la mosca (cajas) y papel que desempeña cada tipo de neurona en DROSOPHILA-LANG (cursiva). La flecha roja es la sinapsis $KC \rightarrow MBON$, cuya fuerza w guarda un número; la dopamina (línea discontinua) la modifica.

Todas estas neuronas existen en el conectoma de FlyWire con su tipo celular anotado. La tabla 1 da cuántas hay de cada tipo, un dato que más adelante limitará el tamaño de los programas (Apartado 5.3).

3. EL LENGUAJE

²En la mosca real, la dopamina *debilita* la sinapsis $KC \rightarrow MBON$ de las células de Kenyon que respondían al olor en ese momento [7]. DROSOPHILA-LANG simplifica: las PAM suman uno y las PPL1 restan uno, para tener un contador que sube y baja.

TABLA 1. Neuronas de cada tipo en el conectoma FlyWire v783 según sus anotaciones, y uso que les da DROSOPHILA-LANG.

Tipo	Neuronas	Uso en DROSOPHILA-LANG
Células de Kenyon (KC)	5177	lectura de la memoria, pasos intermedios
Locales del cuerno lateral	514	compuertas de las preguntas ?r
Dopaminérgicas PAM	307	sumar uno a la memoria
MBON	96	detectar si la memoria es cero
Dopaminérgicas PPL1	16	restar uno a la memoria

Definición 3.1 (Programa). *Un programa de DROSOPHILA-LANG es una tupla $P = (Q, R, q_0, \delta)$, donde Q es un conjunto finito de estados (glomérulos), R un conjunto finito de variables (engramas) con valores en $\mathbb{N}$, $q_0 \in Q$ el estado inicial (el soplo) y δ asigna a cada estado una secuencia finita de acciones seguida de un desenlace: un salto a otro estado, una pregunta sobre una variable con dos estados posibles, o el final.*

3.1. VARIABLES Y ESTADOS. En cada momento el programa está en un solo estado. Los nombres de estados y variables los elige quien escribe el programa. El programa más corto escribe una letra y termina:

```

~~~> (DA1)           ;; empezar en el paso DA1
(DA1) !'F' x_x        ;; escribir F y terminar

```

3.2. LAS ÓRDENES. La tabla 2 reúne todo lo que se puede escribir. Un paso tiene siempre la misma forma: *su nombre entre paréntesis, lo que hace y cómo termina*. Por ejemplo, (VA3) +alfa ~> (VA2) se lee «en el paso VA3, suma 1 a alfa y ve al paso VA2».

TABLA 2. Las órdenes de DROSOPHILA-LANG y su equivalente en programación.

Escribes	Qué hace	En Python o Java
§ r = 0	crea la variable r a 0	r = 0
~~~> (X)	el programa empieza en el paso X	main
+r, -r	suma 1, resta 1 (sin bajar de 0)	r++, r--
<r	lee el siguiente número de la entrada y lo suma a r	r += input()
!"hola"	escribe un texto fijo	print("hola")
!*	mueve la trompa una vez; el intérprete cuenta cuántas	contador++
~> (Y)	va al paso Y	goto Y
?r ~> (A)   (B)	si $r > 0$ , le resta 1 y va a A; si $r = 0$ , va a B	if / else
x_x	termina el programa	exit()

La pregunta ? es la única forma de tomar una decisión, y solo sabe preguntar una cosa: «¿es mayor que cero?». Todo lo demás (comparar dos números, multiplicar...) se construye con bucles a partir de ella, y el Apartado 5 demuestra que eso basta. Los símbolos siguen la metáfora del olfato: la flecha ondulada ~> es algo que viaja, como un olor; la larga ~~~> es el *soplo* que pone el programa en marcha; los paréntesis rodean el nombre de un glomérulo; y `x_x`, el emoticono de los ojos en cruz, es la muerte del programa.

**3.3. VARIABLES Y DATOS.** Las variables y los datos de entrada son cosas distintas. Los datos forman una *cola* que espera fuera del cerebro, tan larga como se quiera; cada < saca el siguiente. Las variables son lo que el programa recuerda a la vez, y caben como mucho 16, por una razón anatómica que explica el Apartado 4. Con dos variables se puede leer un texto de mil letras: se lee cada letra en una, se vacía y se suma 1 a la otra. Como < suma el dato a lo que ya hubiera, una variable se reutiliza vaciándola antes.

**3.4. UN EJEMPLO COMPLETO: LA SUMA.** El programa `suma.dros` lee dos números, los suma y escribe el resultado. A la derecha, lo mismo escrito en Python.

---

```
$ alfa = 0
$ beta = 0
~~~> (DM1)
(DM1) <alfa <beta ~> (VA2)
(VA2) ?beta ~> (VA3) | (DL5)
(VA3) +alfa ~> (VA2)
(DL5) !"suma: " ?alfa ~> (DA2) | (VM5d)
(DA1) ?alfa ~> (DA2) | (VM7d)
(DA2) !* ~> (DA1)
(VM5d) !'0' ~> (VM7d)
(VM7d) !'\n' x_x
```

---



---

```
alfa += int(input())
beta += int(input())
while beta > 0:
 beta -= 1
 alfa += 1
print("suma: ", end="")
if alfa == 0:
 print("0")
else:
 n = 0
 while alfa > 0:
 alfa -= 1
 n += 1
 print(n)
```

---

Con la entrada (3, 4), la mosca guarda `alfa = 3` y `beta = 4`. El bucle `VA2 ↔ VA3` pasa las cuatro unidades de `beta` a `alfa`, una a una, hasta que `beta` vale 0 y `alfa` vale 7. No existe una orden «sumar»: sumar es pasar unidades de una variable a otra. El lenguaje tampoco puede escribir directamente el valor de una variable, así que el final vacía `alfa` moviendo la trompa de la mosca una vez por unidad; el intérprete cuenta los movimientos y escribe `suma: 7`. La Figura 2 lo dibuja como un diagrama de estados.

Los nombres de los pasos (DM1, VA2...) son glomérulos reales, pero no tienen una función fija: VA2 no «sirve para bucles». Lo que hace un paso lo decide lo que se escribe en su línea; el nombre solo decide qué neurona lo ejecuta. Con nombres inventados, como (BUCLE) o (SUMAR), el programa haría exactamente lo mismo. La gramática completa está en el Apartado A.

## 4. CÓMO SE EJECUTA

**4.1. DEL COMANDO AL RESULTADO.** Un programa se ejecuta con una orden como `./mosca.sh suma.dros --olor "3 4"`. En ella intervienen tres piezas que conviene no mezclar: el *programa* (`suma.dros`, solo texto, la receta), el *lanzador* (`mosca.sh`, un

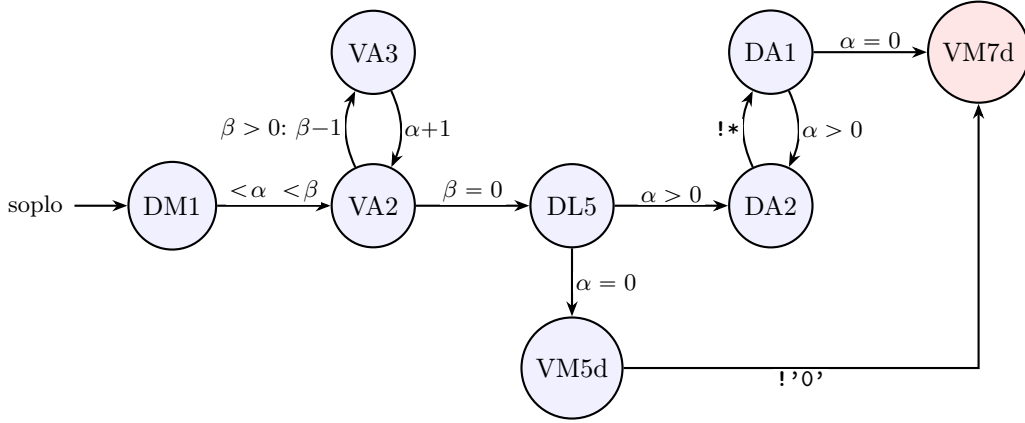


FIGURA 2. Diagrama de estados de `suma.dros`. Cada círculo es un glomérulo; las flechas indican a dónde pasa el control y qué condición o acción lo acompaña. En DL5 se escribe `suma:` antes de la primera pregunta. VM7d escribe el salto de línea y termina.

atajo que activa Python) y el *intérprete* (`drosophila.py`), que es donde está todo lo relacionado con neuronas. La relación es la misma que entre un `script.py` y el programa `python` que lo ejecuta.

FlyWire no proporciona ningún programa: proporciona *datos*, como una hoja de cálculo gigante con cada neurona, su tipo, su posición y sus conexiones. El intérprete los descarga una vez (unos 135 MB) y hace con ellos lo que muestra la Figura 3: lee la receta, elige neuronas reales para cada pieza, las conecta y simula el cerebro paso a paso.

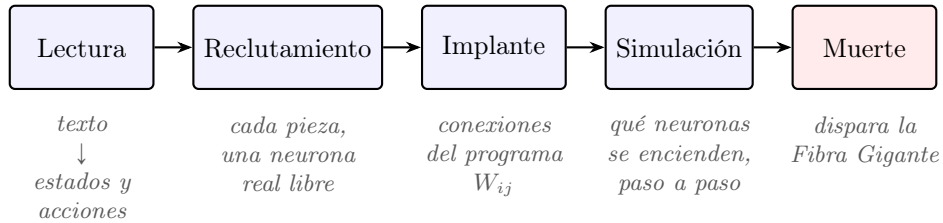


FIGURA 3. Etapas del intérprete, de izquierda a derecha, y lo que produce cada una.

**4.2. NEURONAS REALES, CABLES AÑADIDOS.** Quien ejecuta el programa son siempre neuronas que existen en el mapa. Si dos de ellas ya están conectadas en la mosca, se aprovecha esa conexión; si no, el intérprete añade un cable directo, estén cerca o lejos (en la simulación la distancia no cuenta). En `suma.dros`, de las 84 conexiones que necesita el programa, 8 existen en la mosca y 76 las añade el intérprete. Los datos de FlyWire no se modifican: los cables del programa van en una capa aparte.

**4.3. EL MODELO DE NEURONA.** El intérprete simula el cerebro en tiempo discreto,  $t = 0, 1, 2, \dots$ . En cada paso, cada neurona  $i$  está encendida o apagada,  $s_i(t) \in \{0, 1\}$ , y su estado en  $t + 1$  depende solo de las neuronas que dispararon en  $t$ . Hay dos clases de neuronas: las *reclutadas*, a las que el intérprete asigna un papel en el programa, y las *naturales*, que son todas las demás.

*Conexiones reales.* FlyWire da, para cada par  $j \rightarrow i$ , el número de sinapsis  $c_{ji}$  y el signo  $\varepsilon_j \in \{+1, -1\}$  de la neurona  $j$  (excitadora o inhibidora). El intérprete toma estos

datos del repositorio que acompaña al modelo de Shiu *et al.* [3], conserva las conexiones con al menos cinco sinapsis y normaliza cada peso por la entrada total de la neurona que lo recibe,  $\tilde{w}_{ij} = \varepsilon_j c_{ji} / \sum_k c_{ki}$ . Así  $\sum_j |\tilde{w}_{ij}| = 1$ , y  $\sum_j \tilde{w}_{ij} s_j(t)$  es la fracción neta de las sinapsis de entrada de  $i$  que están activas.

*Neuronas naturales.* Cada una acumula esa entrada en un potencial con fuga (en cada paso conserva la mitad del anterior) y dispara cuando supera el umbral 0,08; después queda dos pasos en silencio (periodo refractario). Es una versión muy simplificada del modelo de integración y disparo de Shiu *et al.*: sirve para ver cómo reacciona el cerebro al programa, no para reproducir tasas de disparo reales.

*Neuronas reclutadas.* El programa se instala como un circuito extra con pesos enteros  $W_{ij}$  que fija el intérprete. Cada neurona reclutada calcula

$$h_i(t) = \underbrace{\sum_j W_{ij} s_j(t) + b_i + I_i(t)}_{\text{programa}} + \underbrace{\kappa \sum_j \tilde{w}_{ij} s_j(t)}_{\text{cerebro}}, \quad \kappa = \frac{1}{8}, \quad (4.1)$$

donde  $b_i \in \{-\frac{1}{2}, -\frac{3}{2}\}$  es un sesgo e  $I_i(t)$  un estímulo externo entero (el soplo inicial y la entrada olfativa), y dispara con probabilidad

$$\Pr[s_i(t+1) = 1] = \sigma(\beta h_i(t)) = \frac{1}{1 + e^{-\beta h_i(t)}}, \quad \beta = 64. \quad (4.2)$$

El parámetro  $\beta$  es la inversa de una temperatura que mide el ruido: cuando  $\beta \rightarrow \infty$  la neurona se vuelve una puerta de umbral exacta, que dispara si y solo si  $h_i > 0$ , como la neurona de McCulloch y Pitts [4]. El factor  $\kappa$  reduce la influencia del cerebro natural sobre el programa a un octavo; el Lema 5.2 muestra que, en el límite  $\beta \rightarrow \infty$ , eso basta para que nunca cambie una decisión, y el Corolario 5.3 acota la probabilidad de que ocurra con  $\beta$  finito.

**4.4. LA MEMORIA.** Cada variable  $r$  es el peso  $w_r \in \mathbb{N}$  de una sinapsis real  $KC_r \rightarrow MBON_r$ , la misma que usa la mosca para recordar un olor (tabla 3). Si  $PAM_r$  o  $PPL1_r$  disparan en el paso  $t$ , el peso cambia según  $w_r(t+1) = \max(0, w_r(t) + s_{PAM_r}(t) - s_{PPL1_r}(t))$ , así que sube o baja de uno en uno y nunca es negativo. Para leerla, el programa enciende  $KC_r$ ; la neurona  $MBON_r$ , con sesgo  $-\frac{1}{2}$ , recibe  $w_r - \frac{1}{2}$  y dispara si y solo si  $w_r \geq 1$ . Una sola neurona responde así a la pregunta «¿es  $r \neq 0$ ?», pero no dice cuánto vale  $r$ : para saberlo hay que vaciar la variable contando, que es lo que hace el final de la suma. En la simulación de la suma, la variable alfa usa una célula de Kenyon de tipo KCg-s2, la MBON05, la PAM08 y la PPL108.

TABLA 3. Las neuronas de una variable  $r$  y la operación que realiza cada una.

Neurona o sinapsis	Papel	Operación
$KC_r \rightarrow MBON_r$ , peso $w_r$	guarda el valor	$r$
$PAM_r$	suma uno al peso	$r++$
$PPL1_r$	resta uno al peso, sin bajar de 0	$r--$
$KC_r$	lee la variable	—
$MBON_r$	dispara si $w_r \geq 1$	$r > 0$

**4.5. CÓMO ELIGE LAS NEURONAS EL INTÉRPRETE.** El intérprete elige cada neurona por su *familia* y por sus *conexiones reales*, nunca por proximidad:

- (1) cada pieza necesita un tipo de neurona: un paso, la de su glomérulo; una variable, una KC, una MBON, una PAM, una PPL1 y un receptor olfativo; una pregunta, cuatro neuronas del cuerno lateral;
- (2) dentro de cada familia, prefiere neuronas que ya estén conectadas entre sí, para añadir menos cables (para una variable, la pareja  $KC \rightarrow MBON$  con más sinapsis);
- (3) si no hay ninguna, toma la primera libre en un orden fijo, así que el mismo programa elige siempre las mismas neuronas.

Cada neurona tiene un solo papel. Es lo mismo que hace un compilador al reservar memoria para las variables: un sitio libre para cada una, sin repetir, y un error si no caben. Como la mosca solo tiene 16 neuronas PPL1, no caben más de 16 variables.

**4.6. DOS CAPAS DE ACTIVIDAD.** En cada paso se actualizan a la vez las 138 639 neuronas, y en el cerebro conviven dos capas de actividad:

- **El programa.** Una única señal recorre las neuronas de control por los cables del programa, modifica variables al pasar por las PAM y PPL1, y en cada pregunta consulta una MBON para elegir el siguiente paso. En la suma son 51 neuronas.
- **El resto del cerebro.** Cada neurona del programa excita o inhibe a sus vecinas reales, y se forman ondas de actividad que se apagan en pocos pasos. Con la suma de 3 y 4 llegan a encenderse unas 600 neuronas a la vez, y 3658 distintas en total.

La segunda capa no altera la primera: por (4.1), la parte del programa siempre está al menos a  $\frac{1}{2}$  del umbral, y la del cerebro nunca supera  $\frac{1}{8}$  (Lema 5.2). Por eso el resultado no depende de la actividad natural.

**4.7. CÓMO TERMINA.** La salida de números usa la motoneurona MN9, que en la mosca extiende la probóscide, la «trompa» con la que come [8]. Cada  $!*$  la dispara una vez y el intérprete cuenta los disparos, como quien cuenta con palitos. El final normal es la Fibra Gigante, la neurona que en la mosca desencadena el salto de huida [9]: cuando dispara, el intérprete para. También se detiene si el programa cae en silencio («coma»), si repite exactamente un estado anterior («ciclo», un bucle infinito) o si agota el número máximo de pasos.

## 5. ¿POR QUÉ ES TURING-COMPLETO?

Un lenguaje es *Turing-completo* si puede calcular cualquier cosa que calcule un ordenador, suponiendo que tiene tiempo y memoria de sobra. La forma habitual de demostrarlo no es construir un ordenador entero, sino mostrar que el lenguaje puede imitar otro sistema del que ya se sabe que es universal. Aquí ese sistema son las máquinas de Minsky, porque sus instrucciones coinciden casi una a una con las de DROSOPHILA-LANG.

**Definición 5.1 (Máquina de Minsky).** Una máquina de Minsky es un par  $M = (Q, \mu)$ , donde  $Q$  es un conjunto finito de estados con un estado inicial  $q_0$ , y  $\mu$  asigna a cada  $q \in Q$  una instrucción sobre registros  $r_1, \dots, r_m \in \mathbb{N}$ :

- $INC(r, q')$ : suma 1 al registro  $r$  y pasa al estado  $q'$ ;
- $JZDEC(r, q', q'')$ : si  $r > 0$ , le resta 1 y pasa a  $q'$ ; si  $r = 0$ , pasa a  $q''$ ;

- **HALT:** *se detiene.*

Una configuración es un par  $(q, \mathbf{x})$  con  $q \in Q$  y  $\mathbf{x} \in \mathbb{N}^m$  el contenido de los registros; cada instrucción define la configuración siguiente.

**5.1. MÁQUINAS DE MINSKY.** Minsky demostró en 1967 que esto basta, incluso con solo dos registros, para imitar cualquier máquina de Turing y, por tanto, cualquier ordenador [10]. Korec construyó además en 1996 una máquina de Minsky *universal* concreta,  $U_{32}$ , de 32 instrucciones y 8 registros: una sola máquina fija que, si recibe en sus registros la descripción de otra máquina y sus datos, la ejecuta [11]. Es lo mismo que hace un ordenador con un programa cualquiera.

**5.2. LA TRADUCCIÓN.** Cada instrucción de Minsky tiene su equivalente directo en DROSOPHILA-LANG (tabla 4): los registros son variables, los estados son pasos, y la instrucción JZDEC es exactamente la pregunta  $?$ . El programa `suma.dros` es, de hecho, una máquina de Minsky con algo de entrada y salida alrededor: su núcleo son  $VA2 : JZDEC(\beta, VA3, DL5)$  y  $VA3 : INC(\alpha, VA2)$ .

TABLA 4. Cómo se traduce cada instrucción de Minsky a DROSOPHILA-LANG y qué neuronas la ejecutan.

Minsky	DROSOPHILA-LANG	Neuronas
$INC(r, q')$	$(q) \text{ +r } \leadsto (q')$	el paso enciende la PAM de $r$
$JZDEC(r, q', q'')$	$(q) \text{ ?r } \leadsto (q') \mid (q'')$	el circuito de la Figura 4
HALT	$(q) \text{ x_x}$	la Fibra Gigante

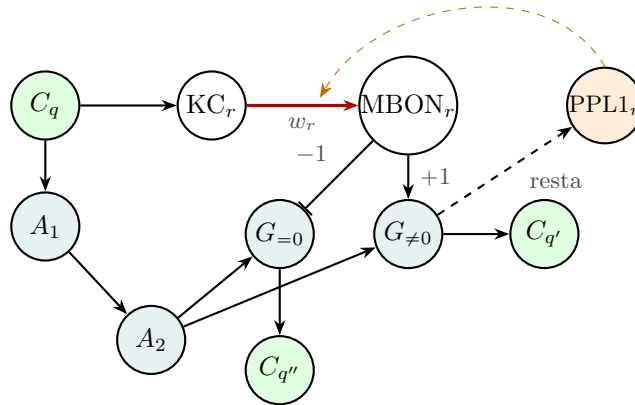


FIGURA 4. El circuito que ejecuta una pregunta  $?\mathbf{r} \leadsto (A) \mid (B)$ . La neurona del paso ( $C_q$ ) enciende la KC, que lee la variable: la MBON solo responde si la variable no es cero. A la vez, la señal pasa por dos neuronas de retardo ( $A_1, A_2$ ) para llegar a las dos compuertas justo cuando llega la respuesta de la MBON. Si la MBON respondió, se enciende la compuerta de arriba, que resta 1 mediante la PPL1 y pasa el turno al paso A ( $C_{q'}$ ); si no, se enciende la de abajo y pasa el turno al paso B ( $C_{q''}$ ). Las compuertas y los retardos (en turquesa) son neuronas locales del cuerno lateral.

Formalmente, la traducción asigna a cada registro  $r$  su engrama ( $KC_r$ ,  $MBON_r$ ,  $PAM_r$ ,  $PPL1_r$  y el peso  $w_r$  de la Apartado 4.4) y a cada estado  $q$  una neurona de control  $C_q$  con sesgo  $-\frac{1}{2}$ .  $INC(r, q')$  se convierte en las conexiones  $C_q \rightarrow PAM_r$  y  $C_q \rightarrow C_{q'}$ ;  $HALT$ , en  $C_q \rightarrow GF$ , donde  $GF$  es la Fibra Gigante; y  $JZDEC(r, q', q'')$ , en el circuito de la Figura 4. En él, las dos compuertas son puertas de umbral: en el límite  $\beta \rightarrow \infty$ ,  $G_{\neq 0}$  dispara si y solo si  $A_2 + MBON_r - \frac{3}{2} > 0$  (una puerta Y) y  $G_{=0}$  si y solo si  $A_2 - MBON_r - \frac{1}{2} > 0$  (« $A_2$  y no  $MBON_r$ »).

Queda comprobar que el circuito se comporta como la máquina ideal. Podrían fallar tres cosas: que el ruido o el resto del cerebro inviertan una decisión, que queden dos estados activos a la vez, o que una pregunta lea una variable antes de que termine de escribirse. El primer punto es el único que necesita un cálculo.

**Lema 5.2 (Margen).** *Para toda neurona reclutada  $i$  y todo paso  $t$ ,  $|h_i(t)| \geq \frac{3}{8}$ .*

*Demostración.* En (4.1), los pesos  $W_{ij}$ , los estímulos  $I_i$  y los pesos de memoria  $w_r$  son enteros, y el sesgo  $b_i$  es un entero menos  $\frac{1}{2}$ . La parte del programa es, por tanto, de la forma  $n + \frac{1}{2}$  con  $n \in \mathbb{Z}$ , y su valor absoluto es al menos  $\frac{1}{2}$ . Como  $\sum_j |\tilde{w}_{ij}| = 1$  y  $s_j \in \{0, 1\}$ , la parte del cerebro cumple  $|\kappa \sum_j \tilde{w}_{ij} s_j| \leq \kappa = \frac{1}{8}$ . Luego  $|h_i| \geq \frac{1}{2} - \frac{1}{8} = \frac{3}{8}$ .  $\square$

**Corolario 5.3 (Probabilidad de error).** *Si un programa usa  $N$  neuronas reclutadas durante  $\tau$  pasos, la probabilidad de que alguna se comporte de forma distinta que en el límite  $\beta \rightarrow \infty$  es como mucho*

$$\tau N \sigma\left(-\frac{3\beta}{8}\right) \leq \tau N e^{-3\beta/8}. \quad (5.1)$$

*Demostración.* Por el Lema 5.2 y (4.2), cada neurona se equivoca en cada paso con probabilidad como mucho  $\sigma(-3\beta/8)$ ; la probabilidad de la unión de  $\tau N$  sucesos es como mucho la suma de sus probabilidades.  $\square$

Con  $\beta = 64$ , la cota por neurona y paso es  $\sigma(-24) \approx 3.8 \times 10^{-11}$ . Para `suma.dros` con entrada (3, 4), que usa  $N = 51$  neuronas durante  $\tau = 87$  pasos, (5.1) da una probabilidad total de error menor que  $2 \times 10^{-7}$ . La cota crece con  $\tau$ , así que una ejecución mucho más larga necesita un  $\beta$  mayor para el mismo margen. Los otros dos puntos se siguen de la construcción:

- **Un solo estado activo.** En  $INC$  y en los saltos, cada neurona de control conecta con una única neurona de control sucesora; en una pregunta, los dos sucesores cuelgan de las compuertas, y estas son excluyentes, porque una exige  $MBON_r = 1$  y la otra  $MBON_r = 0$ . Por inducción sobre  $t$ , en cada paso hay a lo sumo una  $C_q$  activa.
- **Escribir antes de leer.** El cambio de  $w_r$  se aplica en el paso siguiente al disparo de la  $PAM_r$  o  $PPL1_r$ , y los retardos  $A_1$ ,  $A_2$  hacen que la siguiente lectura de  $KC_r$  llegue después de ese cambio.

**5.3. EL CEREBRO ES FINITO.** Un ordenador teórico tiene memoria ilimitada, pero el cerebro de la mosca tiene un número finito de neuronas, y cada pieza del programa gasta alguna. Los dos recursos que antes se agotan son las neuronas  $PPL1$ , una por variable, de las que hay 16; y las neuronas locales del cuerno lateral, cuatro por pregunta, de las que

hay 514. Por tanto, un programa puede tener como mucho 16 variables y 128 preguntas ( $514/4 = 128,5$ ). El resto de tipos de neurona sobra.

**Teorema 5.4.** DROSOPHILA-LANG es *Turing-completo*.

*Demostración.* Toda máquina de Minsky con como mucho 16 registros y 128 instrucciones JZDEC se puede traducir a DROSOPHILA-LANG con la tabla 4, y cabe en el cerebro. Por el Lema 5.2 y los dos puntos que lo siguen, en el límite  $\beta \rightarrow \infty$  el programa traducido pasa por los mismos estados que la máquina, con sus variables siempre iguales a los registros, y dispara la Fibra Gigante exactamente cuando la máquina se detiene. No hace falta que quepa cualquier máquina: basta con que quepa una universal. La  $U_{32}$  de Korec tiene 8 registros y 32 instrucciones, así que cabe; DROSOPHILA-LANG puede simularla, y con ella cualquier cálculo. La memoria ilimitada que exige la definición está en la fuerza de las conexiones, que puede crecer sin límite.  $\square$

Como todo lenguaje Turing-completo, DROSOPHILA-LANG hereda también el problema de la parada [12]: no existe un método general para saber de antemano si un programa terminará. El intérprete detecta los casos fáciles (silencio o un estado repetido), pero no todos.

## 6. RESULTADOS

**6.1. PROGRAMAS DE EJEMPLO.** La tabla 5 resume la ejecución de los programas de ejemplo. Todos producen la salida correcta. `longitud.dros` (Apartado B) lee un texto, en el que cada byte llega como una concentración de olor, y cuenta sus bytes.

TABLA 5. Ejecución de los programas de ejemplo. *Pasos*: pasos hasta la muerte; *reclut.*: neuronas reclutadas; *despiertas*: neuronas naturales que dispararon al menos una vez; *reales*: conexiones del programa que ya existían en el conectoma.

Programa	Entrada	Salida	Pasos	Reclut.	Despiertas	Reales	Tiempo
<code>efe.dros</code>	—	F	3	13	76	1/6	<0.01 s
<code>hola.dros</code>	—	¡Hola, Mundo!	17	27	860	1/83	0.01 s
<code>suma.dros</code>	0, 0	suma: 0	26	51	3248	8/84	0.01 s
<code>suma.dros</code>	3, 4	suma: 7	87	51	3658	8/84	0.04 s
<code>suma.dros</code>	12, 30	suma: 42	427	51	3970	8/84	0.22 s
<code>suma.dros</code>	200, 300	suma: 500	4525	51	3971	8/84	2.26 s
<code>longitud.dros</code>	mosca	bytes: 5	2747	55	2173	10/98	1.24 s

**6.2. DÓNDE VIVE EL PROGRAMA.** La Figura 5 muestra, sobre una vista frontal del cerebro, las 51 neuronas reales que recluta `suma.dros`. 47 de ellas están anotadas como del hemisferio derecho: cuando no hay una neurona conectada que preferir, el intérprete elige primero las del lado derecho. Los efectores (motoneuronas, DN_a02, MN9 y la Fibra Gigante) quedan en la parte ventral.

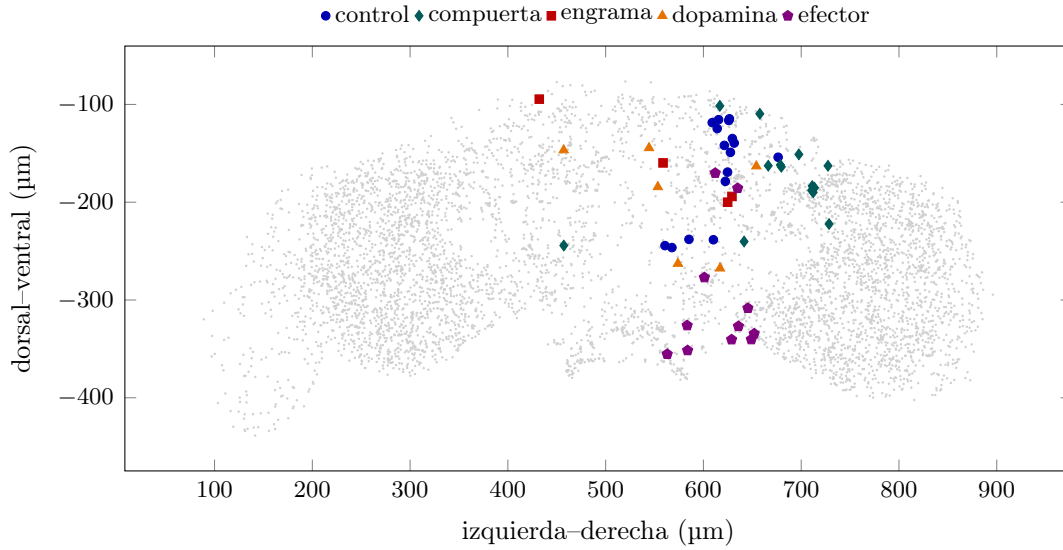


FIGURA 5. Vista frontal del cerebro de la mosca (puntos grises: una muestra aleatoria de 6000 neuronas en su posición real) y las 51 neuronas que recluta `suma.dros`, coloreadas por su papel. El eje vertical crece hacia arriba, hacia la parte dorsal.

**6.3. EL CEREBRO REACCIONA.** Aunque el cerebro natural no decide nada, sí responde al programa: cada vez que dispara una neurona reclutada, sus conexiones reales excitan o inhiben a otras. La Figura 6 muestra cuántas neuronas naturales disparan en cada paso de `suma.dros` con entrada (3,4). Se distinguen las fases del programa. Los picos aparecen al empezar, al oler cada número y en cada vuelta del bucle de transferencia: en todos esos momentos disparan neuronas de proyección, que en el conectoma conectan con cientos de células de Kenyon.

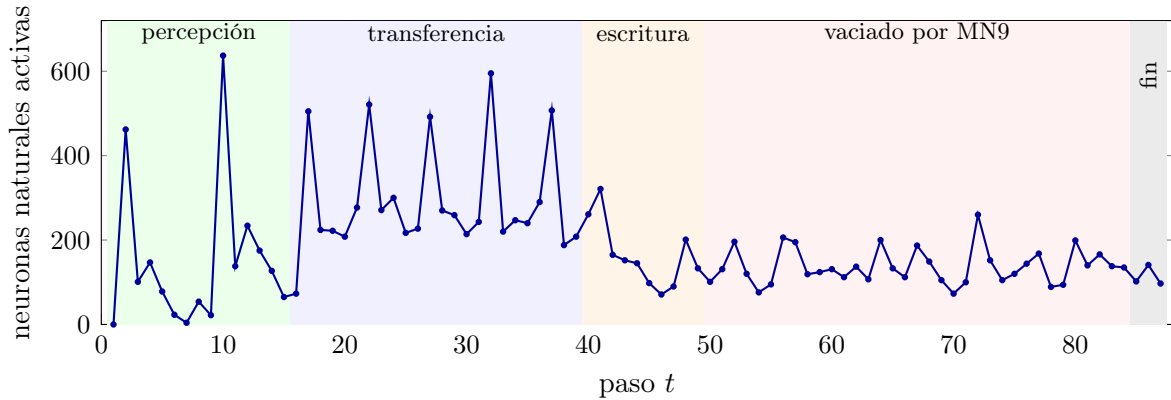


FIGURA 6. Neuronas naturales activas en cada paso de `suma.dros` con entrada (3,4). Las bandas marcan las fases del programa, obtenidas de qué neuronas reclutadas disparan en cada paso. En total despertaron 3658 neuronas distintas.

**6.4. COMPROBACIÓN DE LA DEMOSTRACIÓN.** El Apartado 5 demuestra que la traducción funciona para cualquier máquina de Minsky que quepa en el cerebro; este experimento lo comprueba sobre el mapa real. El guion `minsky.py` genera máquinas

aleatorias, de 2 a 12 estados y de 1 a 3 registros, las traduce con la tabla 4, las ejecuta y compara los registros finales con una simulación directa en Python. De 3000 máquinas generadas, 1333 se detienen en menos de 2000 pasos, y las 1333 dieron el mismo resultado en el cerebro que en la simulación directa.

El límite del Apartado 5.3 también se comprobó ejecutándola: un programa con 128 preguntas `?r` se instala y termina, y uno con 129 falla porque el cerebro se queda sin neuronas del cuerno lateral.

**6.5. ROBUSTEZ FRENTE AL RUIDO.** Las neuronas reclutadas son estocásticas: por (4.2), disparan con una probabilidad que depende de  $\beta$ , y con  $\beta$  pequeño se equivocan a menudo. El valor por defecto es  $\beta = 64$ , para el que el Corolario 5.3 garantiza un error despreciable; este experimento mide qué pasa por debajo. La Figura 7 muestra cuántas de 40 ejecuciones de `suma.dros` con entrada (3, 4), cada una con una semilla de ruido distinta, escriben `suma: 7`. La transición es brusca: todas aciertan para  $\beta \geq 24$ , la mitad para  $\beta = 16$  y ninguna para  $\beta \leq 12$ . El fallo típico es que la Fibra Gigante dispare antes de tiempo y la mosca «huya» sin terminar la cuenta: así acaban 39 de los 40 fallos con  $\beta = 12$  y 18 de los 20 con  $\beta = 16$ . La cota (5.1) es coherente con esto: para  $\beta = 24$  vale  $87 \cdot 51 \cdot e^{-9} \approx 0,55$ , que ya no garantiza nada aunque en la práctica todas las ejecuciones acierten, mientras que para  $\beta = 64$  es menor que  $2 \times 10^{-7}$ .

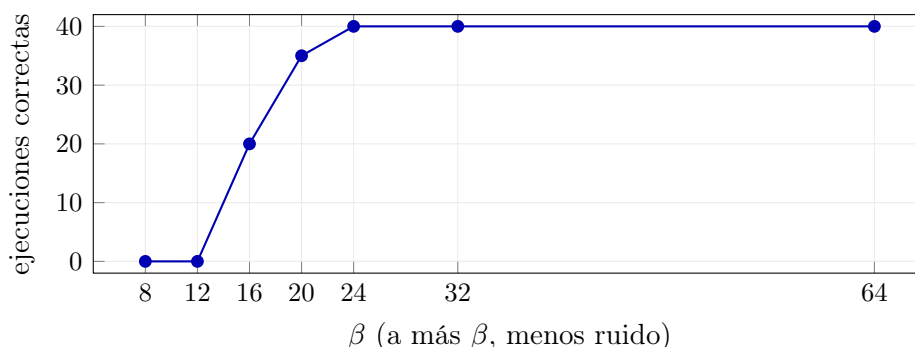


FIGURA 7. Ejecuciones correctas de `suma.dros` (entrada (3, 4)) sobre 40 semillas, para cada valor de  $\beta$ . Cuanto mayor es  $\beta$ , menos ruido. El valor por defecto es  $\beta = 64$ .

## 7. REFLEXIÓN

**7.1. ¿CALCULA LA MOSCA?** Hay que separar lo que es real de lo que añade el intérprete. Son reales las neuronas, con su identidad, su tipo y su posición, y son reales las 2 700 513 conexiones por las que se propagan las olas de actividad de la Figura 6. Pero el circuito que hace el cálculo es casi todo añadido: de las 84 conexiones que usa `suma.dros`, solo 8 existen en la mosca. Las demás funcionan como un transgén, un cableado que la mosca no tiene. Y el resto del cerebro, por diseño, solo le llega al programa atenuado por  $\kappa = \frac{1}{8}$ , y el Lema 5.2 garantiza que no cambia ninguna decisión.

La respuesta es que no: la mosca no suma. Su cerebro pone el escenario y los actores; el programa pone el guion. Que solo 8 de cada 84 conexiones sean reales mide la distancia entre un cerebro que evolucionó para oler, comer y huir y uno diseñado para contar.

**7.2. PROGRAMAR NO ES ENTRENAR.** En la mosca viva, las neuronas PAM se activan con premios, como el azúcar, y las PPL1 con castigos, como una pequeña descarga, y así la mosca aprende qué olores le convienen. DROSOPHILA-LANG no usa premios ni castigos: el intérprete conecta la neurona de un paso directamente con la PAM, y cuando la señal pasa, la PAM se enciende. Sumar 1 es como darle a una sola neurona de dopamina una dosis mínima y exacta de azúcar, sin azúcar de verdad.

Entrenar sería repetir premios y castigos hasta que la mosca aprendiera algo por sí sola, de forma lenta e imprecisa; es como enseñar un papel a un actor. Programar en DROSOPHILA-LANG es decir exactamente qué neurona se enciende después de cuál; es como mover una marioneta con hilos. Los pesos de las conexiones reales nunca cambian, los cables y umbrales del programa los fija el intérprete por diseño, y lo único que cambia durante la ejecución son las variables, de uno en uno y solo cuando una instrucción lo ordena. Se toma prestado el *mecanismo* con el que la mosca aprende, no su significado.

**7.3. ¿Y EN UNA MOSCA VIVA?** Una parte sí sería posible. En los laboratorios se escribe en la memoria de la mosca asociando un olor con un premio o un castigo, y también se pueden activar neuronas concretas con luz mediante técnicas genéticas (optogenética). Así se podría cambiar la fuerza de una conexión  $KC \rightarrow MBON$ , aunque sin la precisión de sumar exactamente uno. Lo que no se puede es crear los 76 cables que le faltan a la suma: en un cerebro vivo no se conectan a voluntad dos neuronas cualesquiera. La única forma de «ejecutar» el programa sería un ordenador externo que leyera las neuronas y activara con luz la siguiente, pero entonces quien calcularía sería el ordenador. Por eso DROSOPHILA-LANG es un experimento mental hecho con datos reales: muestra qué haría falta para que ese cerebro calculara.

**7.4. ¿DÓNDE ESTÁ EL PROGRAMA?** En un ordenador, el programa y la máquina son cosas separadas: el mismo procesador ejecuta cualquier programa que se le cargue. En un cerebro no ocurre así. Lo que la mosca «sabe hacer» está escrito, en buena parte, en su propio cableado, moldeado por la evolución y retocado por lo que aprende: el conectoma es a la vez la máquina y el programa. DROSOPHILA-LANG lo pone de manifiesto: para que la mosca ejecute un programa nuestro no basta con darle datos, hay que recablearla. En ese sentido, el lenguaje no enseña a la mosca a calcular; la usa como material para construir un ordenador distinto.

**7.5. QUÉ HACE FALTA PARA CALCULAR.** El Apartado 5 deja una lección general. Para calcular cualquier cosa basta un programa finito que sepa sumar uno a unos números, preguntar si son cero y restarles uno. Cualquier sistema capaz de eso es Turing-completo, esté hecho de neuronas, de transistores o de montones de piedras. Lo difícil no es la universalidad, sino la eficiencia: como DROSOPHILA-LANG cuenta de uno en uno, sumar 200 y 300 le cuesta 4525 pasos.

**7.6. LA ANATOMÍA COMO GRAMÁTICA.** En un lenguaje corriente, los límites los pone la memoria del ordenador. En DROSOPHILA-LANG los pone la anatomía: no puede haber más de 16 variables porque la mosca solo tiene 16 neuronas PPL1, ni más de 128 preguntas porque solo tiene 514 neuronas locales en el cuerno lateral.

**7.7. LIMITACIONES.**

- **La plasticidad está simplificada.** En la mosca, la dopamina debilita la conexión  $KC \rightarrow MBON$  de las células de Kenyon activas [7]; aquí una familia suma y la otra resta.

- **El modelo de neurona es mínimo.** Sirve para ver cómo se propaga la actividad, pero no está ajustado para reproducir la actividad real, como sí lo está el de Shiu *et al.* [3].
- **La salida es pobre.** Un programa solo puede escribir caracteres fijados en el código o un número. No puede calcular una letra y escribirla, así que la manipulación de texto se limita a programas que leen texto y devuelven números, como `longitud.dros`.

## 8. CONCLUSIONES

*We can only see a short distance ahead, but we can see plenty there that needs to be done.*³

– A. M. Turing [13]

DROSOPHILA-LANG es un lenguaje esotérico cuyos programas se ejecutan sobre el conectoma completo de la mosca de la fruta. Su sintaxis traduce la anatomía: los estados son glomérulos, la memoria es una sinapsis que la dopamina modifica, las preguntas las responden las MBON y el cuerno lateral, y el programa termina con la huida de la mosca. Hemos demostrado que es Turing-completo simulando máquinas de Minsky, con un argumento que tiene en cuenta que el cerebro es finito, y lo hemos comprobado con 1333 máquinas aleatorias ejecutadas sobre el conectoma real.

Quedan abiertas varias extensiones: elegir las neuronas del programa para maximizar las conexiones que ya existen en el conectoma, dejar que la actividad natural participe en las decisiones en lugar de solo observarlas, y sustituir el modelo de neurona por uno ajustado a datos fisiológicos.

## APÉNDICE A. GRAMÁTICA

Gramática completa de DROSOPHILA-LANG en notación EBNF. Se admiten también alias Unicode: § por \$,  $\approx$  por  $\sim$ ,  $\rightsquigarrow$  por  $\leadsto$  y † por x_x.

---

```

programa = { linea } ;
linea = [sentencia] [";" comentario] salto ;
sentencia = directiva | engrama | soplo | glomerulo ;
directiva = "%" ("cepa" | "temperatura") valor ;
engrama = "$" nombre ["=" natural] ;
soplo = "~~~>" glom ;
glomerulo = glom { accion } desenlace ;
glom = "(" nombre ")" ;
accion = "+" nombre | "-" nombre | "<" nombre | "!" emision ;
emision = caracter | cadena | natural | "*" ; (* natural de 0 a 255 *)
desenlace = "~>" glom
 | "?" nombre "~>" glom "|" glom
 | "x_x" ;
nombre = letra { letra | digito | "_" } ; (* distinto de x_x *)

```

---

La directiva `%cepa` fija la semilla del ruido con el nombre de una cepa de laboratorio de la mosca, como Canton-S o w1118, y `%temperatura` el valor de  $1/\beta$ . Un engrama usado sin declarar empieza en 0; si no hay soplo, el programa empieza en el primer estado.

³«Solo podemos ver un poco más allá, pero ya vemos mucho por hacer» (traducción propia).

## APÉNDICE B. PROGRAMAS DE EJEMPLO

LISTADO 1. `hola.dros`: escribe un saludo en UTF-8.

---

```
%cepa w1118
~~~> (DM1)
(DM1) !"¡Hola, Mundo!\n" ~> (DA1)
(DA1) x_x
```

---

LISTADO 2. `longitud.dros`: cuenta los bytes de un texto (se ejecuta con `./mosca.sh longitud.dros --olor-texto "mosca"`).

---

```
$ n = 0                                ;; contador de bytes
~~~> (DM1)
(DM1) <c ~> (VA2) ;; oler el siguiente byte
(VA2) ?c ~> (VA3) | (DL5) ;; olor nulo: fin del texto
(VA3) ?c ~> (VA3) | (VA4) ;; olvidar el resto
(VA4) +n ~> (DM1) ;; recordar un byte más
(DL5) !"bytes: " ?n ~> (DA2) | (VM5d)
(DA1) ?n ~> (DA2) | (VM7d) ;; vaciar el contador...
(DA2) !* ~> (DA1) ;; ...moviendo la probóscide
(VM5d) !'0' ~> (VM7d)
(VM7d) !'\n' x_x
```

---

## APÉNDICE C. CÓMO EJECUTARLO

Desde la carpeta del proyecto, tras instalarlo una vez con `bash instalar.sh`:

---

<code>./mosca.sh efe.dros</code>	# escribe F
<code>./mosca.sh suma.dros --olor "3 4"</code>	# suma: 7
<code>./mosca.sh longitud.dros --olor-texto "mosca"</code>	# bytes: 5
<code>./mosca.sh suma.dros --olor "2 1" --traza</code>	# neuronas encendidas, paso a paso
<code>./mosca.sh suma.dros --reparto</code>	# qué neurona real hace cada papel
<code>python visor3d.py suma.dros --olor "3 4"</code>	# visor 3D del cerebro
<code>python envivo.py</code>	# editor con ejecución en tiempo real

---

TABLA 6. Opciones principales del intérprete.

Opción	Para qué
<code>--olor "3 4"</code>	datos de entrada: números
<code>--olor-texto "..."</code>	datos de entrada: un texto, letra a letra
<code>--traza</code>	muestra qué neuronas se encienden en cada paso
<code>--reparto</code>	muestra qué neurona real hace cada papel
<code>--anatomia</code>	cuenta cuántas conexiones del programa existen en la mosca
<code>--puertas</code>	comprueba las puertas lógicas hechas con neuronas
<code>--temperatura T</code>	cambia el ruido ( $T = 1/\beta$ ; 0 = sin ruido)
<code>--cerebro sintetico</code>	usa un cerebro pequeño inventado, sin datos de FlyWire
<code>-e "código"</code>	ejecuta código escrito en la propia orden

## REFERENCIAS

- [1] Dorkenwald, S. et al. Neuronal wiring diagram of an adult brain. *Nature* 634.8032 (2024), págs. 124-138. DOI: 10.1038/s41586-024-07558-y.
- [2] Schlegel, P. et al. Whole-brain annotation and multi-connectome cell typing of *Drosophila*. *Nature* 634.8032 (2024), págs. 139-152. DOI: 10.1038/s41586-024-07686-5.
- [3] Shiu, P. K. et al. A *Drosophila* computational brain model reveals sensorimotor processing. *Nature* 634.8032 (2024), págs. 210-219. DOI: 10.1038/s41586-024-07763-9.
- [4] McCulloch, W. S. y Pitts, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics* 5.4 (1943), págs. 115-133. DOI: 10.1007/bf02478259.
- [5] Kurtovic, A., Widmer, A. y Dickson, B. J. A single class of olfactory neurons mediates behavioural responses to a *Drosophila* sex pheromone. *Nature* 446.7135 (2007), págs. 542-546. DOI: 10.1038/nature05672.
- [6] Aso, Y. et al. The neuronal architecture of the mushroom body provides a logic for associative learning. *eLife* 3 (2014). DOI: 10.7554/elife.04577.
- [7] Hige, T. et al. Heterosynaptic Plasticity Underlies Aversive Olfactory Learning in *Drosophila*. *Neuron* 88.5 (2015), págs. 985-998. DOI: 10.1016/j.neuron.2015.11.003.
- [8] Gordon, M. D. y Scott, K. Motor Control in a *Drosophila* Taste Circuit. *Neuron* 61.3 (2009), págs. 373-384. DOI: 10.1016/j.neuron.2008.12.033.
- [9] Allen, M. J., Godenschwege, T. A., Tanouye, M. A. y Phelan, P. Making an escape: Development and function of the *Drosophila* giant fibre system. *Seminars in Cell & Developmental Biology* 17.1 (2006), págs. 31-41. DOI: 10.1016/j.semcdb.2005.11.011.
- [10] Minsky, M. L. *Computation: Finite and Infinite Machines*. Englewood Cliffs, NJ: Prentice-Hall, 1967.
- [11] Korec, I. Small universal register machines. *Theoretical Computer Science* 168.2 (1996), págs. 267-301. DOI: 10.1016/s0304-3975(96)00080-1.
- [12] Turing, A. M. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* s2-42.1 (1937), págs. 230-265. DOI: 10.1112/plms/s2-42.1.230.
- [13] Turing, A. M. Computing Machinery and Intelligence. *Mind* 59.236 (1950), págs. 433-460. DOI: 10.1093/mind/LIX.236.433.